SemEval 2027 Task 9: Analysis of Multimodal Framing in the News

Arbeitsblatt für den Forschungsblock

Heinrich-Heine-Universität Düsseldorf

Woche 4

Name: **Teamrolle:** **Datum:** 03.11.2026

1 Grundidee

- **Unser Einstieg: Subtask 1, Englisch.** Pro Absatz werden null, ein oder mehrere der 14 Framing-Labels vorhergesagt. Kein Frame entspricht einem Nullvektor [1].
- Framing bezeichnet die hervorgehobene Perspektive, nicht nur das Thema. Beispiel: `Economic` fragt nach wirtschaftlichen Aspekten (ja oder nein).
- Ein Sigmoid-Modell liefert zunächst 14 Werte zwischen 0 und 1. Schwellen erzeugen den **binären 14-dimensionalen Vektor**, kein Softmax mit nur einem Gewinner.

2 Mögliche Baselines

- **Startvorschlag: TF-IDF + 14 binäre logistische Regressionen**, eine pro Label. TF-IDF beschreibt Texte über gewichtete Worthäufigkeiten. Zusätzlich eine konstante Referenz testen.
- Vergleichsmodell: gemeinsamer Textencoder mit 14 Sigmoid-Ausgängen und binärer Kreuzentropie pro Label. Ob es besser ist, entscheidet die Evaluation.
- Ein gemeinsames 14-Output-Modell ist *nicht* dasselbe wie eine eigene Klasse je Labelkombination (Label Powerset). Seltene Kombinationen erschweren letzteren Ansatz.

3 Embedding-basierter Ansatz

- Trainingsabsätze mit einem vortrainierten Embedding-Modell in Vektoren umwandeln. Zunächst etwa die mit `Political` annotierten Beispiele untersuchen.
- Pro Label einen Mittelwert-Prototyp bilden. Optional innerhalb des Labels clustern und mehrere Repräsentanten vergleichen. Vektoren für den Kosinusvergleich normieren.
- Für jeden neuen Absatz *jedes* Label anhand seiner Prototypen prüfen. Eine auf Validierungsdaten gewählte Schwelle erlaubt mehrere oder keine Labels.

4 Erweiterung: Synthetische Trainingsdaten

- Erst nach der echten Baseline: zusätzliche Trainingsbeispiele oder Prototypen testen, insbesondere für seltene Labels. Labelqualität und Wettbewerbsvorgaben prüfen.
- Synthetische Beispiele getrennt kennzeichnen und keine Testdaten zur Generierung nutzen. Nutzen auf denselben echten, zurückgehaltenen Daten messen, nicht auf künstlichen Testbelegen.

5 Erweiterung: Bilder in Text übersetzen

- Pipeline-Idee: Bild → Beschreibung durch ein Vision-Language-Modell, optional OCR → Klassifikation. Artikeltext kann als Kontext hinzukommen.
- Ziel von Subtask 2 sind die **Frames des Bildes**, nicht einfach die des Artikels. Variante 2a nutzt keine Gold-Textlabels, 2b stellt sie bereit [1].
- Beschreibungen auf Auslassungen und erfundene Details prüfen. Caption allein und Caption mit Artikelkontext vergleichen und die Übertragung des Textmodells gesondert evaluieren.

Leitfrage: Welche einfache Pipeline liefert zuerst eine belastbare Vergleichsbasis? Für alle Versuche gelten derselbe dokumentweise Split und das Evaluationsprotokoll auf Seite 3.

Hauptaufgaben und Aufgaben für heute

Name: **Arbeitspaket:** **Datum:** 03.11.2026

1 Fünf Arbeitspakete

Jede Person übernimmt ein Arbeitspaket und dokumentiert ein überprüfbares Ergebnis.

	Arbeitspaket	Auftrag für heute	Angestrebtes Ergebnis
☐	**Literatur und Hypothesen**	Zwei bis drei relevante Arbeiten lesen, Labeldefinitionen und Methoden vergleichen. Mit Card et al. [2] beginnen.	Quellen mit Kernaussage und zwei testbare Hypothesen.
☐	**Daten und Labels**	Labelhäufigkeiten, Kein-Frame-Fälle und Dubletten prüfen. Gemeinsamen dokumentweisen Split festlegen.	Statistik, Labelreihenfolge und gespeicherte Split-IDs.
☐	**Text-Baseline**	TF-IDF + 14 binäre logistische Regressionen auf dem gemeinsamen Split starten. Konstante Referenz ergänzen.	Startbefehl, Konfiguration und erste Vorhersagen.
☐	**Embeddings und Prototypen**	Vortrainiertes Modell wählen, Trainingsprototypen und Kosinus-Schwellen testen. Clustering optional.	Modellname, Schwellenregel und erste Vorhersagen.
☐	**Evaluation und Fehleranalyse**	Offiziellen Scorer und Ausgabeformat prüfen, Baselines vergleichen und Fehlertypen sammeln.	Micro-/Macro-F1, drei Fehlerbeispiele, Bildpipeline als Ausblick.

2 Agenda

Tätigkeit	Erwartetes Zwischenergebnis
Gemeinsamer Start: Daten, Split und Pakete klären.	Gemeinsame Vergleichsbasis festhalten.
Forschungsfrage und erwartetes Ergebnis festlegen.	Hypothese oder Entscheidungskriterium.
Recherche, Analyse oder erster technischer Versuch.	Quellen, Notizen, Code oder Statistik.
Ergebnis anhand des gemeinsamen Protokolls prüfen.	Artefakt, Konfiguration und Evidenz.
Ergebnisse im Team vorstellen und diskutieren.	Ergebnis, Problem und Empfehlung.
Gemeinsame Entscheidungen und nächste Schritte.	Zuständigkeit und nächster Termin.

3 Meine Dokumentation

Meine konkrete Forschungsfrage oder Hypothese:

Wichtigste Erkenntnisse / Evidenz:

Probleme / offene Fragen:

Mein konkretes Ergebnis (Link, Datei, Statistik, Pipeline oder Literaturübersicht):

Meine Empfehlung an das Team:

☐ Ergebnis vorgestellt ☐ Artefakt abgelegt ☐ Nächsten Schritt und Zuständigkeit festgelegt

Referenz und Evaluationsprotokoll

1 Die 14 Framing-Labels

Die hervorgehobene Perspektiven sind Beispiele für die 14 Framing-Labels. Mehrere Perspektiven können gleichzeitig vorliegen. Ein Stichwort allein genügt nicht. Für Grenzfälle gelten die offiziellen Richtlinien und Beispiele [1, 2].

Label (Lesefassung)	**Hervorgehobene Perspektive**
Capacity and resources	Verfügbare Mittel, Personal und Handlungskapazität.
Crime and punishment	Straftaten, Strafverfolgung und Bestrafung.
Cultural identity	Traditionen, kulturelle Werte und Zugehörigkeit.
Economic	Kosten, Nutzen und wirtschaftliche Auswirkungen.
External regulation and reputation	Internationale Regeln, Beziehungen und Ansehen.
Fairness and equality	Gerechtigkeit und Verteilung von Rechten oder Lasten.
Health and safety	Gesundheit und körperliche Unversehrtheit.
Legality, constitutionality and jurisprudence	Rechtslage, Verfassung und gerichtliche Auslegung.
Morality	Moralische oder religiöse Maßstäbe und Pflichten.
Policy prescription and evaluation	Maßnahmen, Lösungsvorschläge und ihre Bewertung.
Political	Politischer Wettbewerb, Macht und Parteistrategien.
Public opinion	Öffentliche Einstellungen und gesellschaftliche Zustimmung.
Quality of life	Wohlergehen und alltägliche Lebensbedingungen.
Security and defense	Schutz vor Bedrohungen, Sicherheit und Verteidigung.

Lesefassung mit korrigiertem „resources“. Im Code exakt die Label-IDs des Starter-Kits verwenden. „Kein Frame“ ist kein zusätzliches inhaltliches Frame-Label.

2 Gemeinsamer Versuchsstandard

- **Split:** Alle Absätze eines Artikels bleiben zusammen. Train-/Validation-IDs und Zufallsseed speichern. Identische oder nahezu identische Artikel auf Datenlecks prüfen.
- **Training:** TF-IDF nur auf Training fitten und Prototypen nur daraus bilden. Vortrainierte Encoder sind erlaubt, sofern Daten- und Wettbewerbsregeln dies zulassen.
- **Auswahl:** Modelle und Schwellen nur mit Validation auswählen. Testdaten bleiben unangetastet. Validierungsergebnisse als solche kennzeichnen, nicht als unabhängigen Test.
- **Messung:** Offiziellen Scorer nutzen: Micro-F1 primär, Macro-F1 ergänzend [1]. Dessen `None`-Behandlung prüfen und Nullvektoren nicht unbemerkt ausschließen.
- **Nachweis:** Datenversion, Split, Modell, Parameter, Schwellen und Startbefehl dokumentieren und pro Label Fehler ansehen. Dokument-ID und Absatz-Offsets für die Ausgabe erhalten.

3 Methodische Einordnung

- Die Prototypen-Baseline benötigt kein zusätzliches Klassifikatortraining, ist aber nicht modellfrei. Ein vortrainiertes Embedding-Modell erzeugt die Vektoren.
- Bei synthetischen Daten erzeugt ein Sprachmodell Beispieltexte. Das Embedding-Modell berechnet deren Vektoren. Das ist noch kein Training eines eigenen Embedding-Modells.

Paket 1: Literatur und Hypothesen

Ziel heute: Zwei nachvollziehbare Hypothesen, die unser Team mit den Baselines testen kann. Du musst heute weder ein Modell trainieren noch die gesamte Literatur abdecken.

Startpunkt und Voraussetzungen

Browser und Zugriff auf die unten verlinkten Texte reichen. Starte unabhängig von den Trainingsdaten. Alle neuen Ablagepfade auf diesen Anleitungsseiten sind **Vorschläge, die heute anzulegen sind**. Relative Pfade beziehen sich immer auf den Stammordner des Repositorys.

Jetzt starten: Schritt für Schritt

- [ ] **1. Aufgabe eingrenzen.** Öffne die offizielle Task-Beschreibung. Notiere Eingabe, Ziel und Auswertung für Subtask 1. Halte fest, welche Ideen nur Subtask 2 betreffen.
- [ ] **2. Labels verstehen.** Lies in Card et al. (2015) die Beschreibung der Frame-Taxonomie. Wähle drei leicht verwechselbare Labels. Erkläre ihren Unterschied mit eigenen kurzen Beispielen. Kennzeichne selbst formulierte Beispiele als Illustration, nicht als Gold-Annotation.
- [ ] **3. Eine zweite Arbeit vergleichen.** Öffne SemEval-2023 Task 3. Konzentriere dich auf Framing. Notiere Unterschiede zu unserer Absatzaufgabe, insbesondere Vorhersageeinheit, Daten, Baselines und Metriken. Übertrage Resultate nicht ungeprüft.
- [ ] **4. Eine kleine Vergleichstabelle schreiben.** Pro Arbeit: Quelle mit Link, Forschungsfrage, Methode, Daten, Evaluation, eine Stärke, eine Grenze und eine konkrete Idee für unser Projekt. Belege Aussagen mit Abschnitt oder Seite.
- [ ] **5. Zwei testbare Hypothesen ableiten.** Form: „Wenn wir X gegenüber Baseline Y ändern, erwarten wir Z in Metrik M, weil …“. Beispiel: Ein Prototypenmodell erkennt paraphrasierte Frames besser als TF-IDF. Das ist eine Hypothese, kein vorweggenommenes Ergebnis.
- [ ] **6. An das Team übergeben.** Besprich mit Paket 3 und 4, welche Hypothese mit den heutigen Mitteln testbar ist. Gib Paket 5 die drei Labelabgrenzungen als Prüffragen für die Fehleranalyse.

Ergebnis ablegen

`docs/week4/literatur.md`: Vergleich von mindestens zwei Arbeiten, klickbare Quellen, zwei Hypothesen und drei Labelabgrenzungen. Eine dritte Arbeit ist optional.

Fertig, wenn

- [ ] Zwei Arbeiten mit Fundstellen ausgewertet.
- [ ] Jede Hypothese nennt Baseline, Änderung und Messgröße.
- [ ] Eine konkrete Empfehlung wurde an die Modellpakete übergeben.

Falls du blockiert bist

Wenn ein Volltext fehlt, nutze den PDF-Link auf der ACL-Seite oder zunächst den anderen Text. Markiere reine Abstract-Aussagen als vorläufig. Ohne Daten kannst du die Literaturübersicht vollständig erstellen.

Optional danach: Literatur zu Label-Prototypen oder synthetischen Trainingsdaten suchen. Nicht zusätzlich einen eigenen Modellansatz implementieren.

Paket 2: Daten, Labels und gemeinsamer Split

Ziel heute: Eine geprüfte englische Datenbasis und ein fester Split, den Paket 3, 4 und 5 identisch nutzen.

Vorhanden und noch zu bauen

Vorhanden: Loader, Labelreihenfolge, Statistikbefehl und fünf Loader-Tests. **Noch zu implementieren:** Auswahl der Datenquellen und Export des dokumentweisen Splits.

Jetzt starten: Schritt für Schritt

- [ ] **1. Datenzugang klären.** Folge data/README.md. Lade nur über euren autorisierten Zugang herunter. Prüfe, welche englischen 2027-Dateien tatsächlich vorliegen. Ältere 2023-Daten bleiben zunächst getrennt. Der Loader filtert Sprache und Jahr *nicht* automatisch.

- [ ] **2. Loader prüfen.** Im Repository-Stammordner in PowerShell ausführen. Voraussetzung ist Python ab 3.11. Der Loader benötigt keine zusätzlichen Pakete.

```
$env:PYTHONPATH = "src"
python -m unittest discover -s tests -v
python -m semeval_framing.cli --help
```

  Danach ausführen, den Platzhalter durch den geprüften Datenordner ersetzen:

```
python -m semeval_framing.cli "PFAD_ZUM_EN_ORDNER"
```

- [ ] **3. Format und Statistik prüfen.** Erwarteter Dateiname:
  `subtask-1-annotations-with-paragraphs.txt`.
  Prüfe zehn Zeilen einschließlich leerer und mehrfacher Labels. Notiere Anzahl der Dokumente, Absätze und Labels sowie den Kein-Frame-Anteil. `--json` liefert maschinenlesbare Statistik.

- [ ] **4. Quellen zusammenführen und Dubletten klären.** Nutze `read_st1_file` für eine explizite Dateiliste. Gleiche Dokument-IDs und Offsets ab, prüfe identische Texte und protokolliere die Bereinigung. Der bestehende Loader meldet doppelte Schlüssel als Fehler. Nicht durch Umbenennen umgehen.

- [ ] **5. Split implementieren und speichern.** Lege `src/semeval_framing/split.py` neu an. Für scikit-learn die virtuelle Umgebung aus Paket 3 einrichten. Startvorschlag: GroupShuffleSplit, 20 % Validierungsdokumente, Seed 42, Gruppen nach Dokument-ID. Prüfe disjunkte Dokumentmengen und Labelhäufigkeiten je Split. Verwandte Dubletten müssen zusammenbleiben.

- [ ] **6. Früh übergeben.** Speichere lokal `data/processed/week4/split.json` mit Dokument-IDs, Quellenliste und Seed sowie `paragraphs.jsonl` mit `document_id`, `begin`, `end`, `text`, `labels`, `split`. Bestätige Paket 3, 4 und 5 denselben Stand. Labelreihenfolge ist `FRAME_LABELS`.

Fertig, wenn

- [ ] Loader-Tests grün, Quellen und Statistik dokumentiert.
- [ ] Keine Dokumentüberschneidung, seltene oder fehlende Trainingslabels gemeldet.
- [ ] `docs/week4/daten.md` enthält Datenversion, Split-Regel und Übergabehinweis.

Falls du blockiert bist

Ohne Zugang: Loader-Tests ausführen und Split mit erfundenen Beispielen testen. Das als Funktionstest, nicht als Wettbewerbsergebnis melden. Bei abweichendem Dateiformat erst den Parser klären. Rohdaten, Goldlabels und lokale Exporte nicht ins öffentliche Repository hochladen.

Paket 3: Erste Text-Baseline

Ziel heute: Ein reproduzierbarer TF-IDF-Lauf und auswertbare Vorhersagen. Ein CPU-Lauf genügt. Ein Transformer-Modell ist heute keine Voraussetzung.

Voraussetzungen und Status

Du brauchst Paket 2s lokalen Export und Split sowie Paket 5s Vorhersageformat. **Ein Trainingsskript ist noch nicht vorhanden.** Die folgenden Modellschritte sind zu implementieren. Starte mit der Repository-Einrichtung. Ein schlanker Weg in PowerShell:

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e .
.\.venv\Scripts\python.exe -m pip install scikit-learn
```

Eine bestehende virtuelle Umgebung wiederverwenden. Falls `py -3.11` fehlt, eine vorhandene Python-Version ab 3.11 verwenden. Alle weiteren Python-Aufrufe in dieser Umgebung ausführen.

Jetzt starten: Schritt für Schritt

- [ ] **1. Train und Validation laden.** Lade `data/processed/week4/paragraphs.jsonl` und die Split-Datei. Verwende exakt die von Paket 2 freigegebenen Beispiele. Keine eigene zufällige Aufteilung erzeugen.

- [ ] **2. Zielmatrix bauen.** Verwende `MultiLabelBinarizer(classes=FRAME_LABELS)`. Erwartet wird eine Matrix mit 14 Spalten. Leere Labelmengen werden zu Nullvektoren. Fehlende Trainingsklassen und konstante Spalten dokumentieren.

- [ ] **3. Einfache Pipeline implementieren.** Lege `src/semeval_framing/baseline_tfidf.py` neu an. Nutze TfidfVectorizer und OneVsRestClassifier. Startkonfiguration: Wort-Unigramme, `min_df=1`, maximal 30 000 Merkmale, `LogisticRegression(C=1.0, max_iter=1000)`. Fit nur auf Training, danach Validation transformieren und vorhersagen.

- [ ] **4. Referenz und Schwelle festlegen.** Starte mit 0,5 als Schwelle für die vorhergesagten Wahrscheinlichkeiten. Ergänze eine konstante Referenz aus den häufigsten binären Trainingsentscheidungen je Label. Sie kann für seltene Labels ausschließlich Nein ergeben. Beide Systeme identisch auswerten lassen.

- [ ] **5. Vorhersagen exportieren.** Schreibe lokal `outputs/week4/tfidf/validation.tsv` im mit Paket 5 abgestimmten Format. Jede Validierungszeile braucht Dokument-ID, Original-Offsets und vorhergesagte Labels, aber keinen Absatztext. Auch Vorhersagen ohne Frame müssen erhalten bleiben.

- [ ] **6. Lauf reproduzierbar machen.** Baue eine Kommandozeilenschnittstelle für Eingabe, Split und Ausgabe. Dokumentiere den tatsächlich funktionierenden Startbefehl, Paketversionen und Konfiguration. Wiederhole den Lauf einmal und übergib die Vorhersagen an Paket 5.

Fertig, wenn

- [ ] Ein echter Validierungslauf und eine konstante Referenz liegen vor.
- [ ] Ausgabe umfasst genau die freigegebenen Absatzschlüssel.
- [ ] `docs/week4/textbaseline.md` erklärt Aufruf, Datenstand und Grenzen.

Falls du blockiert bist

Bis der Split vorliegt, Pipeline und Export auf erfundenen Beispielen testen. Bei Konvergenzwarnungen Iterationen oder Merkmalszahl anpassen und Änderung notieren. Ein Funktionstest ohne echte Daten ist ein Teilergebnis, keine gemessene Baseline-Leistung.

Paket 4: Embeddings und Label-Prototypen

Ziel heute: Eine zweite Baseline auf demselben Split, ohne einen zusätzlichen Klassifikator zu trainieren. Zunächst ein Prototyp pro Label. Clustering und synthetische Texte sind optional.

Voraussetzungen und Status

Paket 2 liefert Daten und Split, Paket 5 das Ausgabeformat. **Embedding-Skript und Zusatzbibliothek fehlen noch im Repository.** Nutze eine virtuelle Umgebung wie in Paket 3. Installiere darin bei erlaubtem Download:

```
.\.venv\Scripts\python.exe -m pip install sentence-transformers
```

Dokumentiere die installierte Version. Lies die Anleitung zu Text-Embeddings und die Modellkarte von all-MiniLM-L6-v2. Dieses englische Modell ist ein Startvorschlag, kein vorgegebener Wettbewerbssieger.

Jetzt starten: Schritt für Schritt

- [ ] **1. Modell festlegen und Smoke-Test machen.** Lade `sentence-transformers/all-MiniLM-L6-v2` auf CPU. Prüfe mit zwei erfundenen Sätzen, ob Vektoren entstehen. Der erste Aufruf lädt Modellgewichte herunter. Offizielle Texte nicht an einen externen Inferenzdienst senden.

- [ ] **2. Absätze einbetten.** Lege `src/semeval_framing/baseline_prototype.py` neu an. Verwende `encode(..., normalize_embeddings=True)`. Halte Zuordnung zu Absatzschlüsseln fest. Das Startmodell kürzt lange Eingaben standardmäßig nach 256 Wordpieces. Dokumentiere den Anteil betroffener Absätze und die gewählte Behandlung.

- [ ] **3. Prototypen nur aus Training bilden.** Bilde für jedes Label den Mittelwert seiner positiven Trainingsvektoren und normiere ihn erneut. Ein mehrfach gelabelter Absatz darf zu mehreren Prototypen beitragen. Ohne positive Trainingsbeispiele bleibt das Label inaktiv. Melde den Fall, statt Validation beizumischen.

- [ ] **4. Kosinuswerte je Label berechnen.** Für jeden Validierungsabsatz alle aktiven Prototypen vergleichen. Das Skalarprodukt normierter Vektoren ergibt den Kosinuswert. Wähle nicht nur das beste Label. Kosinuswerte sind keine kalibrierten Wahrscheinlichkeiten.

- [ ] **5. Eine gemeinsame Schwelle wählen.** Startvorschlag: 0,2, 0,3, 0,4, 0,5 und 0,6 auf Validation prüfen. Ein Label wird ausgegeben, wenn sein Wert die Schwelle erreicht. Mit Paket 5 die Schwelle nach offiziellem Micro-F1 auswählen und Gleichstände vorab regeln. Diese Werte sind schwellenoptimierte Validierungsergebnisse, keine unabhängigen Testwerte.

- [ ] **6. Export und Übergabe.** Schreibe `outputs/week4/prototype/validation.tsv` im gemeinsamen Format. Dokumentiere Modellrevision, Textkürzung, Schwellenversuche und Trainingshäufigkeiten. Speichere Vektoren nur lokal und gib Paket 5 die Vorhersagen und Laufkonfiguration.

Fertig, wenn

- [ ] Jeder aktive Prototyp stammt nur aus dem Trainingssplit.
- [ ] Alle 14 Labels bleiben in einer festen Reihenfolge, auch inaktive Labels.
- [ ] `docs/week4/embeddings.md` beschreibt Lauf, Schwelle und Grenzen.

Falls du blockiert bist

Ohne Modellzugang oder Internet ein bereits lokal verfügbares Modell verwenden und die Abweichung melden. Andernfalls Kosinusberechnung und Export mit kleinen erfundenen Vektoren testen. Ohne echten Modelllauf keine Qualitätsaussage treffen. Clustering erst nach funktionierender Baseline beginnen.

Paket 5: Evaluation und Fehleranalyse

Ziel heute: Ein geprüftes Ausgabeformat und eine vergleichbare Ergebnistabelle. Du startest vor den Modellläufen und koordinierst die Übergaben mit Paket 2, 3 und 4.

Startpunkt und Voraussetzungen

Öffne die offizielle Evaluation und die Hinweise zum Starter-Kit. Der Scorer ist nicht im Repository enthalten. Zugriff auf das lokale Kit ist nötig, um seinen echten Aufruf zu prüfen. Scorer-Anbindung und Submission-Writer sind noch zu implementieren.

Jetzt starten: Schritt für Schritt

- [ ] **1. Scorer und Anleitung finden.** Prüfe lokal `third_party/official-starter-kit/subtask1/`. Lies dessen README und Hilfe. Notiere Dateiname, Version und Argumente. Verwende den dort dokumentierten Befehl. Ein konkreter Scorer-Aufruf wird hier nicht erfunden.

- [ ] **2. Gemeinsame Schnittstelle sofort mitteilen.** Intern verwenden wir `document_id`, `begin`, `end` und `labels`. Für die Task-Ausgabe werden die Felder durch Tabs getrennt, mehrere Labels ebenfalls. Ohne Frame bleibt das Labelfeld leer. Kein Absatztext, keine selbst erfundenen Labelnamen. Ob eine Kopfzeile erlaubt ist, anhand des Kits prüfen, nicht voraussetzen.

- [ ] **3. Vier kleine Prüffälle auswerten.** Erstelle selbst formulierte Fixtures: identische Gold- und Systemlabels, ein fehlendes Label, ein zusätzliches Label sowie ein beidseitiger Kein-Frame-Fall. Ergänze eine identische gemischte Datei mit positiven und leeren Labels. Prüfe die Behandlung von `None`, leeren Feldern und Absatzschlüsseln.

- [ ] **4. Writer und Formatprüfung implementieren.** Lege `src/semeval_framing/evaluate.py` und `tests/test_evalua` neu an. Prüfe erlaubte Labels, eindeutige Schlüssel, vollständige Zeilenzahl und exakte Offsets. Jeder erwartete Validierungsabsatz muss genau einmal vorkommen.

- [ ] **5. Modelle auf identischer Grundlage vergleichen.** Werte konstante Referenz, TF-IDF und Prototypen auf denselben Validierungs-IDs aus. Tabelle: Modell, Datenversion, Split/Seed, Schwelle, Micro-F1, Macro-F1 und Status. Fehlende Läufe als „nicht durchgeführt“ eintragen, nicht als Nullscore. Prototypen-Schwellenwahl auf Validation transparent ausweisen.

- [ ] **6. Drei Fehler untersuchen und nächste Entscheidung ableiten.** Wähle etwa ein fehlendes, ein falsches und ein verwechseltes Label. Notiere erwartete und vorhergesagte Labels, mögliche Ursache und nächsten Test. Nutze Paket 1s Abgrenzungen. Eine Caption-/OCR-Pipeline nur als optionalen Ausblick skizzieren.

Ergebnis ablegen

`results/week4/metrics.csv`: nur aggregierte Ergebnisse und Konfigurationen.

`docs/week4/evaluation.md`: echter Scorer-Aufruf, Prüffälle, Grenzen und Empfehlung.

`outputs/week4/error_analysis.md`: Fehlerdetails und Textbelege ausschließlich lokal. Keine offiziellen Texte, Goldlabels oder vollständigen Vorhersagen veröffentlichen.

Fertig, wenn

- [ ] Format und Sonderfälle sind geprüft und an das Team kommuniziert.
- [ ] Mindestens die konstante Referenz ist ausgewertet oder der fehlende Scorer klar dokumentiert.
- [ ] Vorliegende Modellläufe sind vergleichbar, drei Fehlerfälle oder Prüffälle sind kommentiert.

Falls du blockiert bist

Ohne Starter-Kit bereits Writer und Fixtures testen. Eine eigene F1-Berechnung ist nur eine vorläufige Diagnose, bis die `None`-Behandlung dem offiziellen Scorer entspricht. Ohne Modellvorhersagen mit der konstanten Referenz arbeiten. Keine Scores erfinden.

Literatur und Quellen

[1] SemEval 2027 Task 9 (o. J.). *Analysis of Multimodal Framing in the News.* Offizielle Task-Beschreibung mit Subtasks, Ausgabeformaten und Evaluation. Zugriff am 3. November 2026.
https://propaganda.math.unipd.it/semeval2027task9/index.html
Zugehörige Label-Liste:
https://propaganda.math.unipd.it/semeval2027task9/data/framing-labels.txt

[2] Card, D., Boydstun, A. E., Gross, J. H., Resnik, P. und Smith, N. A. (2015). *The Media Frames Corpus: Annotations of Frames Across Issues.* In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, S. 438 bis 444. Association for Computational Linguistics.
https://doi.org/10.3115/v1/P15-2072

[3] scikit-learn (o. J.). *Multiclass and multioutput algorithms.* Online-Dokumentation, insbesondere Multi-Label-Klassifikation und One-vs-Rest. Zugriff am 3. November 2026.
https://scikit-learn.org/stable/modules/multiclass.html

[4] Sentence Transformers (o. J.). *Semantic Textual Similarity.* Online-Dokumentation zur Berechnung von Text-Embeddings und Kosinusähnlichkeit. Zugriff am 3. November 2026.
https://www.sbert.net/docs/sentence_transformer/
usage/semantic_textual_similarity.html

[5] Piskorski, J., Stefanovitch, N., Da San Martino, G. und Nakov, P. (2023). *SemEval-2023 Task 3: Detecting the Category, the Framing, and the Persuasion Techniques in Online News in a Multi-lingual Setup.* Proceedings of the 17th International Workshop on Semantic Evaluation.
https://aclanthology.org/2023.semeval-1.317/

[6] Sentence Transformers (o. J.). *all-MiniLM-L6-v2.* Modellkarte mit Hinweisen zur Nutzung und Eingabelänge. Zugriff am 3. November 2026.
https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2